

Miskolci Egyetem
Gépészmérnöki és Informatikai Kar
Matematikai Intézet

2025/2026. tanév II. félév

Párhuzamos eszközök programozása (GEMAK259-B2)

Johnson algoritmus

Jegyzőkönyv

Készítette: Erdélyi Péter (JH3V7T)

Gyakorlatvezető: Piller Imre

Miskolc, 2026. április 30.

TARTALOMJEGYZÉK

1. BEVEZETÉS	1
2. A JOHNSON ALGORITMUS MŰKÖDÉSE	2
2.1. Pszeudokód.....	2
2.2. Az algoritmus főbb lépései	2
3. MÉRÉSI EREDMÉNYEK.....	4
3.1. CPU párhuzamosítás.....	4
3.2. GPU párhuzamosítás	5
4. PÁRHUZAMOSSÁG VIZSGÁLATA	8
4.1. A CPU és GPU mérések összehasonlítása	8
4.2. A Johnson algoritmus GPU skálázhatóságának elemzése.....	9
ÖSSZEGZÉS.....	10
IRODALOMJEGYZÉK.....	11

1. BEVEZETÉS

A Johnson algoritmust megalkotójáról, Donald B. Johnson-ról nevezték el, melyet a szerző 1977-ben publikált először. Az eljárás segítségével egy irányított, élsúlyozott gráfban kiszámíthatók a legrövidebb utak minden csúcspár között. Ez az útkeresés különösen akkor hatékony, ha az élek száma jelentősen kevesebb, mint a maximális élmennyiség, vagyis az útkeresés egy ritka gráfban történik. A végrehajtás lépései között szubrutinként szerepelnek a Bellman-Ford és Dijkstra algoritmusok, ezáltal lehetőség nyílik a negatív élsúlyok kezelésére, ugyanakkor fontos kiemelni, hogy a gráf nem tartalmazhat negatív súlyú kört. A jegyzőkönyv hátralévő része a Johnson algoritmus mérési eredményeiről és annak párhuzamosságáról számol be.

2. A JOHNSON ALGORITMUS MŰKÖDÉSE

2.1. Pszeudokód

A Johnson algoritmus működését az alábbi pszeudokód [1] írja le:

```

1      compute  $G'$ , where  $G'.V = G.V \cup \{s\}$ ,
         $G'.E = G.E \cup \{(s, v) : v \in G.V\}$ , and
         $w(s, v) = 0$  for all  $v \in G.V$ 
2      if BELLMAN-FORD( $G', w, s$ ) == FALSE
3          print "the input graph contains a negative-weight cycle"
4      else for each vertex  $v \in G'.V$ 
5          set  $h(v)$  to the value of  $\delta(s, v)$ 
            computed by the Bellman-Ford algorithm
6      for each edge  $(u, v) \in G'.E$ 
7           $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$ 
8      let  $D = (d_{uv})$  be a new  $n \times n$  matrix
9      for each vertex  $u \in G.V$ 
10         run DIJKSTRA( $G, \hat{w}, u$ ) to compute  $\hat{\delta}(u, v)$  for all  $v \in G.V$ 
11         for each vertex  $v \in G.V$ 
12              $d_{uv} = \hat{\delta}(u, v) + h(v) - h(u)$ 
13      return  $D$ 

```

2.2. Az algoritmus főbb lépései

Az algoritmus leegyszerűsítve a következő lépéseket végzi:

1. Kibővíti az eredeti gráfot egy új csomóponttal (s). Az s csomópontból kiindít egy élt minden más csomópontba, ahol az így keletkező élek súlyát zérusra állítja. Ezáltal létrejön a G' kibővített gráf.
2. Végrehajtja a Bellman-Ford algoritmust a G' kibővített gráfon. Ha a Bellman-Ford hamis értékkel tér vissza, akkor az eredeti gráfban negatív súlyú kör található: ekkor nincs értelme a számítást tovább folytatni, ezért az algoritmus leáll. Ellenkező esetben az algoritmus az élrelaxáció segítségével meghatározott legrövidebb távolságokat ($h(v)$) adja vissza.

3. Átsúlyozza az eredeti G gráfot a $h(v)$ potenciálfüggvényt felhasználva. Ez a lépés biztosítja, hogy az új élsúlyok ($\hat{w}$) nemnegatívak lesznek, ami a háromszögegyenlőtlenségnek köszönhető [1].
4. Lefuttatja a Dijkstra algoritmust az eredeti G gráfon minden csomópontra az új $\hat{w}$ súlyfüggvénnyel. Ennek eredményeképpen megkapjuk a legrövidebb úttávolságokat az átsúlyozott gráfban ($\hat{\delta}(u, v)$).
5. Utolsó lépésben visszaalakítja az eredeti távolságokat, mivel a számításhoz a módosított súlyokat használta. Az algoritmus kimenete a D négyzetes mátrix, amely tartalmazza a valódi legrövidebb úttávolságokat minden egyes csúcspár között.

A Dijkstra negatív élsúlyok esetén nem garantálja a helyes eredményt, ezt a problémát pedig a Johnson algoritmus az élek átsúlyozásával kerüli meg. Ez a folyamat kiküszöböli a negatív élsúlyokat, melynek hatására már alkalmazható a Dijkstra algoritmus. Lényegében a gráfoknak azt a tulajdonságát használja ki, miszerint az átsúlyozás nem változtatja meg a legrövidebb utakat [1].

3. MÉRÉSI EREDMÉNYEK

3.1. CPU párhuzamosítás

A Johnson algoritmus párhuzamosságát már korábban, a „Párhuzamos algoritmusok” tantárgy keretein belül is vizsgáltam. Az ezzel kapcsolatos méréseket és következtetéseket az alábbiakban röviden összefoglalom.

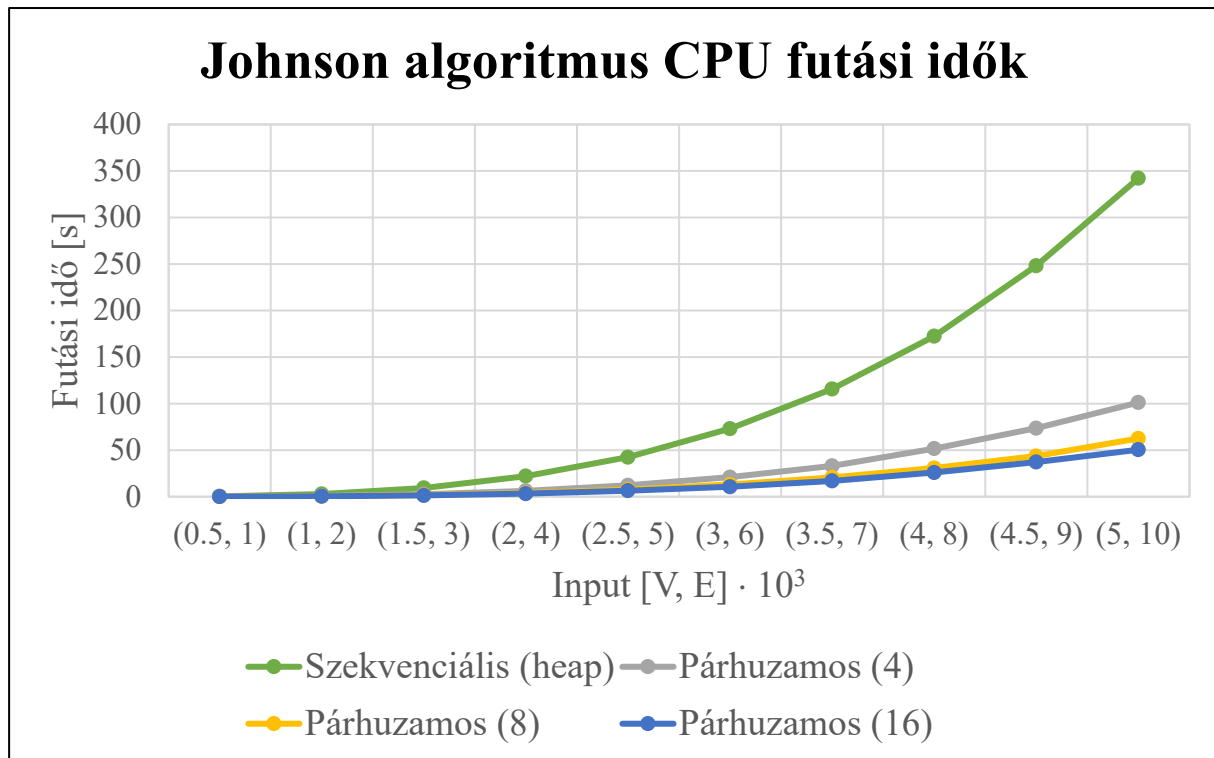
A CPU mérés során kapott eredményeket az 1. táblázat tartalmazza.

Input [V, E]	1 szál [s]	4 szál [s]	8 szál [s]	16 szál [s]
(500, 1 000)	0,3727	0,1097	0,0707	0,0610
(1 000, 2000)	2,8487	0,8150	0,5203	0,4193
(1 500, 3 000)	9,3930	2,6457	1,7057	1,3727
(2 000, 4 000)	22,0980	6,3070	3,9460	3,2593
(2 500, 5 000)	42,5187	12,2413	7,6350	6,4253
(3 000, 6 000)	73,1077	20,9537	13,1103	10,7143
(3 500, 7 000)	115,7980	33,0943	20,7013	16,9290
(4 000, 8 000)	172,4573	51,6973	30,7560	26,0127
(4 500, 9 000)	248,1570	73,6110	43,7090	37,2717
(5 000, 10 000)	342,1603	101,1890	62,5673	50,4723

1. táblázat: A korábbi CPU mérések eredményei

Az „Input” oszlopban tüntettem fel a bemenethez használt adathalmaz nagyságát (csúcsok és élek szerint), míg a többi oszlopban a végrehajtási idők szerepelnek a szálak számának függvényében. Minden mérést három különböző alkalommal végeztem el, így a táblázatba az ezekből képzett átlagok kerültek. Az eredmények kiértékeléséhez tehát a szekvenciális végrehajtás futási idejét hasonlítottam össze a párhuzamos futási idővel 4, 8 és 16 aktív szálat feltételezve.

A soros és párhuzamos végrehajtás futási idejét az alábbi grafikonon (1. ábra) szemlélteti.



1. ábra: A Johnson algoritmus CPU futási idejének vizsgálata

Az előző adatok alapján a következő megállapításokat tettem:

- Az input növelésével a futási idő rendkívül megnövekedett, az algoritmus időbonyolultsága polinomiális jellegűnek látszik.
- A szekvenciális futási idő lényegesen lecsökkent a speciális adatstruktúra (heap) használatával (ezt az implementáció során felismertem, a párhuzamosítást ebből adódóan heap szerkezetet alkalmazva folytattam).
- A szálak számának növelése egyre kisebb gyorsulást eredményezett: kb. háromszoros gyorsítást értem el azzal, ha a feladatot 4 szálon futtattam. További ~ 40%-os javulás volt tapasztalható a futásidőben, ha a végrehajtáshoz 8 db szálát vettem igénybe.
- Az algoritmus végrehajtási ideje 16 aktív szál esetén már nem változott számottevően, ilyenkor a szálak közötti kommunikáció okozta „overhead” már jelentős volt.

3.2. GPU párhuzamosítás

Jelen feladatban a Johnson algoritmus párhuzamosítását videokártyán valósítottam meg az OpenCL szabványt felhasználva. A különböző architektúrák miatt a GPU implementáció némileg eltér az eredeti verziótól, a főbb technikai különbségek:

- Mivel a videokártyának nem lehet beágyazott struktúrát közvetlenül átadni, ezért az eredeti éllistas reprezentáció módosítására volt szükség. Ahhoz, hogy az eszköz a gráfot feldolgozható formában be tudja tölteni, áttértem az ún. CSR formátumra [2], amely ritka gráfok esetén egy kifejezetten memóriahatékony választás.
- A CPU párhuzamosításnál alkalmazott minimum heap adatszerkezetet és thread pool ütemezést teljes mértékben elhagytam.
- Az adatkommunikáció minimalizálása érdekében a Bellman-Ford algoritmus host oldalon maradt, vagyis nem került párhuzamosításra. Ennek elsődleges oka abban rejlik, hogy a Johnson algoritmusnál a szűk keresztmetszetet a Dijkstra futtatása jelenti. Előzetes mérésekből már kiderítettem, hogy a Bellman-Ford párhuzamosítása legtöbbször csak mérési hibahatáron belüli javulást eredményezett.
- A gráf átsúlyozásának lépése korábban szekvenciálisan történt. Ez a művelet a Bellman-Ford-hoz hasonlóan minimális erőforrást igényel a teljes folyamatot tekintve, ugyanakkor az átsúlyozást nem terheli az adatmozgással járó probléma (mert ez indítható közvetlenül a Dijkstra előtt). Éppen ezért ez a lépés egy külön kernelbe került.
- A Dijkstra algoritmusnál a minimum elem kiválasztását lineáris kereséssel oldottam meg a heap adatstruktúra hiányában. Ez a megközelítés közel sem optimális, viszont nagyszerű továbbfejlesztési lehetőséget rejt magában a jövőre nézve.

A GPU mérésnél kapott eredményeket a 2. táblázatban gyűjtöttem össze.

Input [V, E]	Lokális munkacsoportok száma [db]				
	16 [s]	32 [s]	64 [s]	128 [s]	256 [s]
(500, 1 000)	0,0160	0,0210	0,0313	0,0327	0,0343
(1 000, 2000)	0,0440	0,0740	0,0777	0,0820	0,1067
(1 500, 3 000)	0,2033	0,2133	0,2153	0,2267	0,2453
(2 000, 4 000)	0,2353	0,2587	0,2630	0,2950	0,4100
(2 500, 5 000)	0,5503	0,5780	0,6020	0,6067	0,6703
(3 000, 6 000)	0,5727	0,6220	0,6163	0,6887	0,9123
(3 500, 7 000)	1,0817	1,1403	1,1353	1,1693	1,3277
(4 000, 8 000)	1,4697	1,7093	1,7093	1,7877	2,1843
(4 500, 9 000)	3,5230	3,7987	3,7997	3,8677	4,1163
(5 000, 10 000)	3,2553	3,5063	3,5043	3,5930	4,0593

2. táblázat: A GPU mérések eredményei

A skálázhatóság vizsgálatához további méréseket folytattam, ahol a csúcs- és élszámot a korábbi host oldali mérésekhez képest drasztikusan megnőveltem, a lokális munkacsoportok számát pedig az eddigi trendek alapján inkább csökkentettem. Az így kapott végrehajtási időket a 3. táblázatban vázoltam fel.

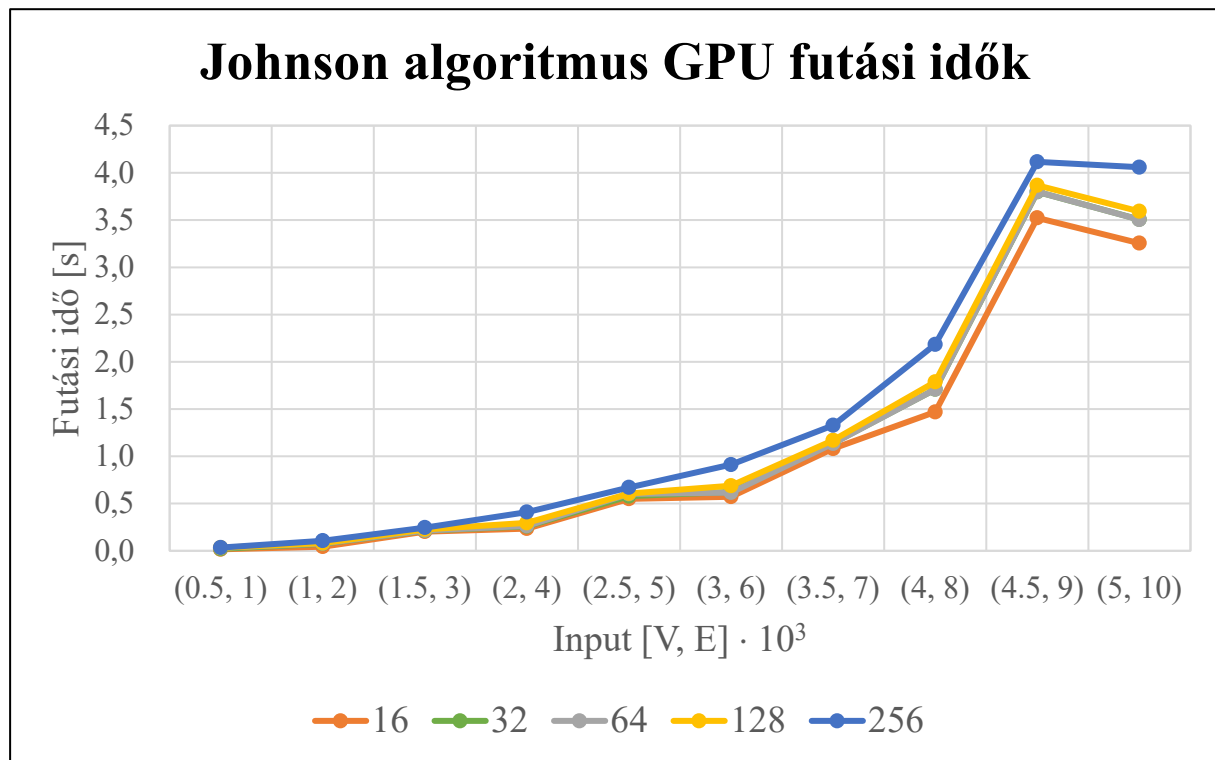
Input [V, E]	Lokális munkacsoportok száma [db]				
	4 [s]	8 [s]	16 [s]	32 [s]	64 [s]
(6 000, 12 000)	4,1850	4,5247	4,8303	5,1710	5,1970
(7 000, 14 000)	6,5583	7,1237	7,5307	7,9197	7,9933
(8 000, 16 000)	9,0233	9,6227	9,9657	10,4880	10,6400
(9 000, 18 000)	15,3140	13,7173	14,2157	15,1343	15,0623
(10 000, 20 000)	21,0317	17,8930	18,3500	19,2860	19,2590
(12 000, 24 000)	31,8637	29,9073	30,5820	31,6007	31,9713
(14 000, 28 000)	48,5787	46,6313	47,7577	49,0620	49,4040
(16 000, 32 000)	71,1883	69,3427	70,3143	71,9713	71,1963
(18 000, 36 000)	114,9633	112,9483	98,9920	100,3730	99,6840
(20 000, 40 000)	143,3537	142,8317	132,6657	134,7433	134,4107

3. táblázat: A GPU mérések eredményei (skálázhatóság)

4. PÁRHUZAMOSSÁG VIZSGÁLATA

4.1. A CPU és GPU mérések összehasonlítása

Első lépésben a processzoron mért korábbi adatokat fogom összehasonlítani a GPU mérések eredményeivel, melynek alapját az 1. és 2. táblázatok adják. A 2. táblázatban kapott értékeket a 2. ábra illusztrálja.



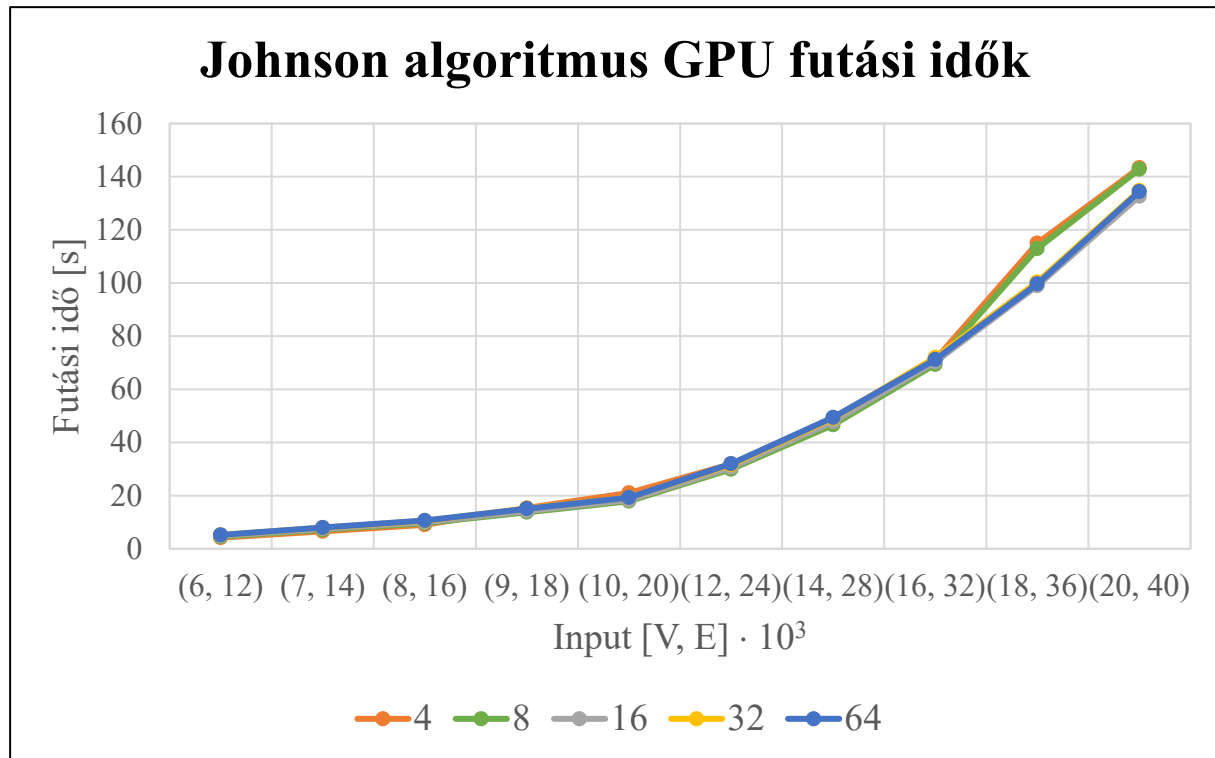
2. ábra: A Johnson algoritmus GPU futási idejének vizsgálata

A mérések szerint levonható következtetések:

- A GPU-val történő párhuzamosítás látványosan hatékonyabb. A jelenlegi OpenCL implementációval közelítőleg 80–100-szoros gyorsítás érhető el a CPU szekvenciális végrehajtásával szemben.
- A lokális munkacsoportok számának növelése a futásidő minimális romlásához vezet.
- A 4500-as és 5000-es csúcsú mérések között rendellenesség lép fel, ugyanis a kevesebb csúcsához nagyobb futásidő tartozik. Ez a jelenség bármilyen lokális munkacsoportszámot választva megfigyelhető.

4.2. A Johnson algoritmus GPU skálázhatóságának elemzése

Az algoritmus skálázhatóságának kiértékeléséhez tekintsük a 3. táblázatban szereplő mérési adatokat, melyeket az alábbi (3. ábra) grafikon mutat be.



3. ábra: A Johnson algoritmus GPU skálázhatóságának elemzése

A nagyméretű adathalmaz tesztelése jónéhány érdekességet fedett fel:

- A 8000 és az alatti csúcstartományban a kisebb munkacsoportszám gyorsabbnak bizonyult. Ez valamelyest igazolta az előző sejtésemet, miszerint a kevesebb munkacsoport gyorsabb végrehajtást eredményez (többek között ezért végeztem ezt a mérést kisebb munkacsoportokkal).
- Egy átfordulási pont vehető észre a 18000-es csomópont körül, ahol már jobb teljesítményt várhatunk a 16-os, 32-es vagy 64-es munkacsoport-beállítás mellett.
- A mérési tartomány jelentős részében azonban a munkacsoport mérete nem befolyásolta döntően a futási időket. Tulajdonképpen ennek a paraméternek csekély hatása van a Johnson algoritmusra nézve, ugyanakkor az optimális választásnak a 8-as vagy 16-os érték tűnik.
- Végeredményben kijelenthető, hogy a Johnson algoritmus GPU implementációja sokkal rugalmasabban skálázható, mint ami a host esetében elmondható.

ÖSSZEGZÉS

Ebben a jegyzőkönyvben a Johnson algoritmus videókártyás implementációjának mérésével foglalkoztam. A feladatban felhasználtam a „Párhuzamos algoritmusok” tantárgyhoz készített programkódot, melynek OpenCL-be történő átültetése okozott némi technikai nehézséget, hiszen az eredetileg alkalmazott gráfos adatstruktúra és párhuzamosítási módszer teljes átalakítására volt szükség. A méréseket két nagyobb szakaszban végeztem: először azon az adathalmazon teszteltem, melyen kezdetben a CPU többszálú teljesítményét tanulmányoztam, ezt követően pedig nagyobb csúcstartományokat vizsgáltam, ahol a GPU skálázhatóságának elemzése volt a célom. Ezen vizsgálatok eredményeképpen megállapítottam egyrészt, hogy a host oldalhoz képest látványos gyorsulást sikerült elérnem a Johnson algoritmus új implementációjával, másrészt azt is, hogy a skálázhatóságra a lokális munkacsoportok mérete nincs döntő befolyással.

IRODALOMJEGYZÉK

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest és C. Stein, *Introduction to Algorithms*, 3. kiadás. Cambridge, MA, USA: MIT Press, 2009.

[2] NVIDIA, „Compressed Sparse Row (CSR) Storage Format”, *NVIDIA Performance Libraries (NVPL) Documentation*, 2024.

https://docs.nvidia.com/nvpl/latest/sparse/storage_format/sparse_matrix.html.

[Megtekintve: 2026. április 30.]